CS 6650 — Homework 10

Distributed Key-Value Databases

Load Test Report

Configurations tested:

- Leader-Follower W=5, R=1
- Leader-Follower W=1, R=5
- Leader-Follower W=3, R=3
- Leaderless W=N=5, R=1

Read/Write ratios: 1/99 · 10/90 · 50/50 · 90/10

1,000 requests per run · 100-key pool · paired-reads queue

Section 1 — Load Test Generator Design

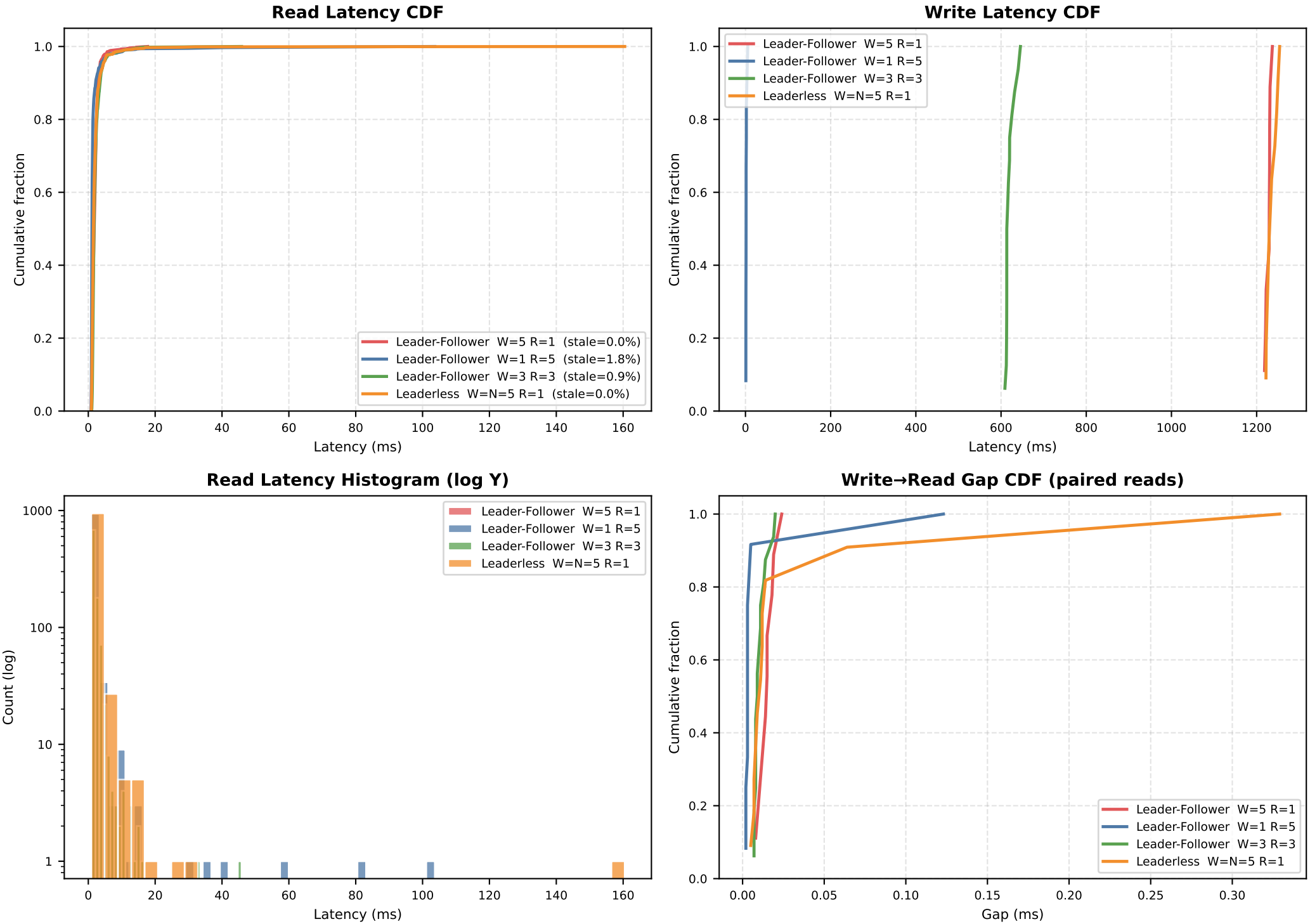
HOW THE LOAD TEST GENERATOR WORKS

```
Key pool:
  100 fixed keys (key-0000 … key-0099). With 1,000 requests per run each key
  is touched ~10 times on average, guaranteeing temporal overlap between reads
  and writes to the same key even without extra logic.

Paired-reads queue (guarantees local-in-time clustering):
  After every successful write to key K with version V:
    → push { key:K, expected_version:V, write_time:T } onto paired_reads deque

  When the next read operation fires:
    → if paired_reads is non-empty: pop and read that exact key from a follower
    → else: pick a random key from the 100-key pool

  This means every write is immediately followed by at least one follower read
  of the same key. The write→read gap in the graphs reflects real elapsed time
  from write ACK to that paired read.

Stale read detection:
  POST /set returns {"version": N}. The client stores known[key] = N.
  When a follower returns version < N (or a 404 for a known written key),
  the read is flagged as STALE.

  Reads always go to FOLLOWER nodes (8002–8005) or NON-COORDINATOR nodes
  (8012–8015) — never to the leader — so leader freshness never masks
  follower staleness.

Why this guarantees frequent same-key reads and writes:
  • Small pool (100 keys) creates natural overlap at high load.
  • Paired-reads queue guarantees at least one follower read per write,
    occurring within milliseconds of the write ACK.
  • The write→read gap histogram shows this directly: for W=1 R=5 gaps
    cluster around 2–5 ms (reads arrive long before 1.2 s propagation),
    for W=5 R=1 gaps cluster around 1,230 ms (reads arrive right after
    the slow synchronous write returns).
```

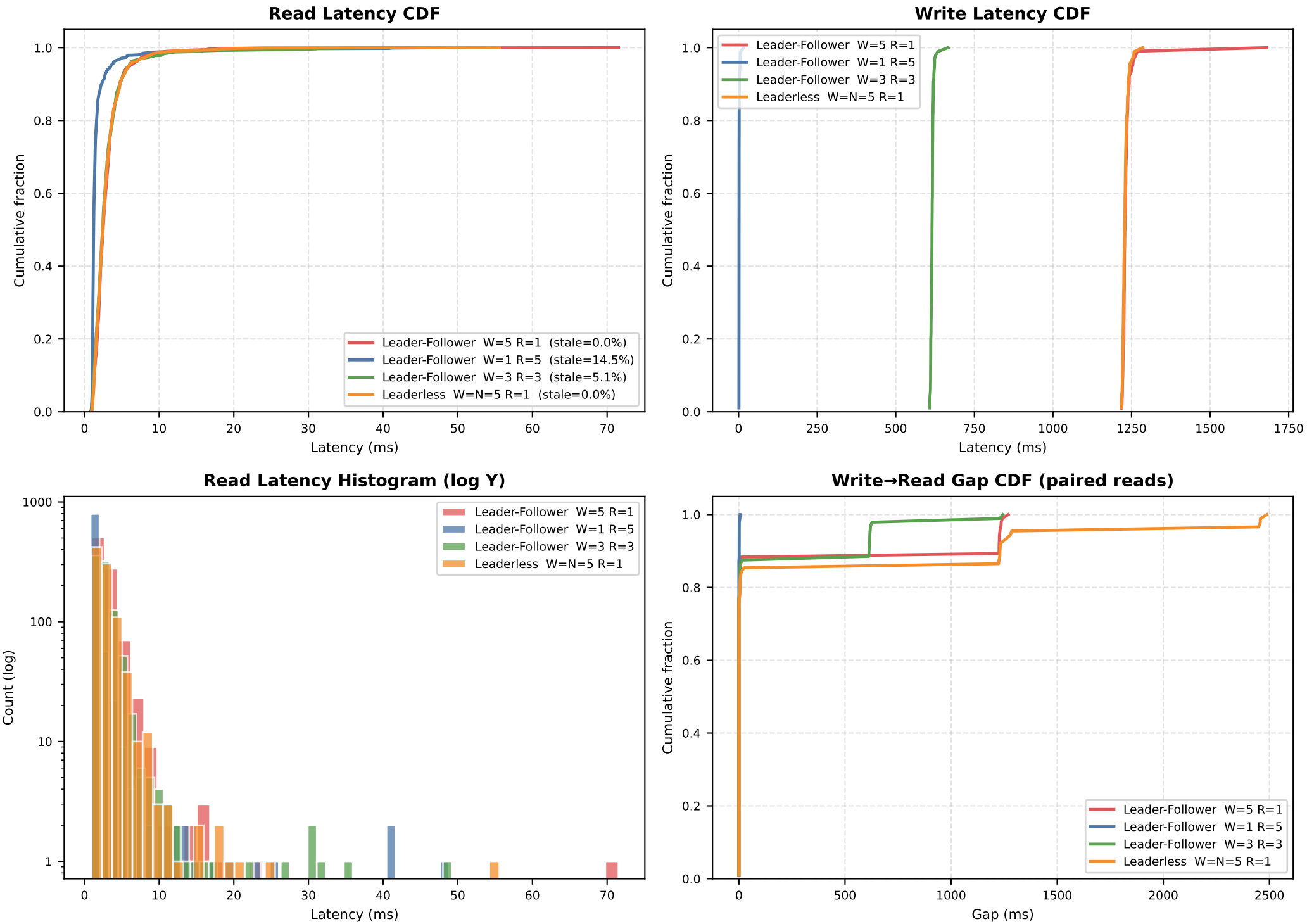
Summary: Average Latency and Stale Read Rates

Config	Ratio	Write avg (ms)	Read avg (ms)	Stale reads	Stale %
Leader-Follower W=5 R=1	1% writes / 99% reads	1228	2.0	0/991	0.0%
Leader-Follower W=5 R=1	10% writes / 90% reads	1236	3.0	0/897	0.0%
Leader-Follower W=5 R=1	50% writes / 50% reads	1232	4.3	0/488	0.0%
Leader-Follower W=5 R=1	90% writes / 10% reads	1237	7.1	0/83	0.0%
Leader-Follower W=1 R=5	1% writes / 99% reads	2	1.8	18/988	1.8%
Leader-Follower W=1 R=5	10% writes / 90% reads	2	1.8	131/906	14.5%
Leader-Follower W=1 R=5	50% writes / 50% reads	2	1.8	479/540	88.7%
Leader-Follower W=1 R=5	90% writes / 10% reads	2	1.4	7/106	6.6%
Leader-Follower W=3 R=3	1% writes / 99% reads	620	2.2	9/984	0.9%
Leader-Follower W=3 R=3	10% writes / 90% reads	616	3.0	46/904	5.1%
Leader-Follower W=3 R=3	50% writes / 50% reads	616	4.3	9/509	1.8%
Leader-Follower W=3 R=3	90% writes / 10% reads	623	5.7	0/104	0.0%
Leaderless W=N=5 R=1	1% writes / 99% reads	1235	2.2	0/989	0.0%
Leaderless W=N=5 R=1	10% writes / 90% reads	1230	3.0	0/911	0.0%
Leaderless W=N=5 R=1	50% writes / 50% reads	1233	4.8	0/521	0.0%
Leaderless W=N=5 R=1	90% writes / 10% reads	1239	7.0	0/78	0.0%

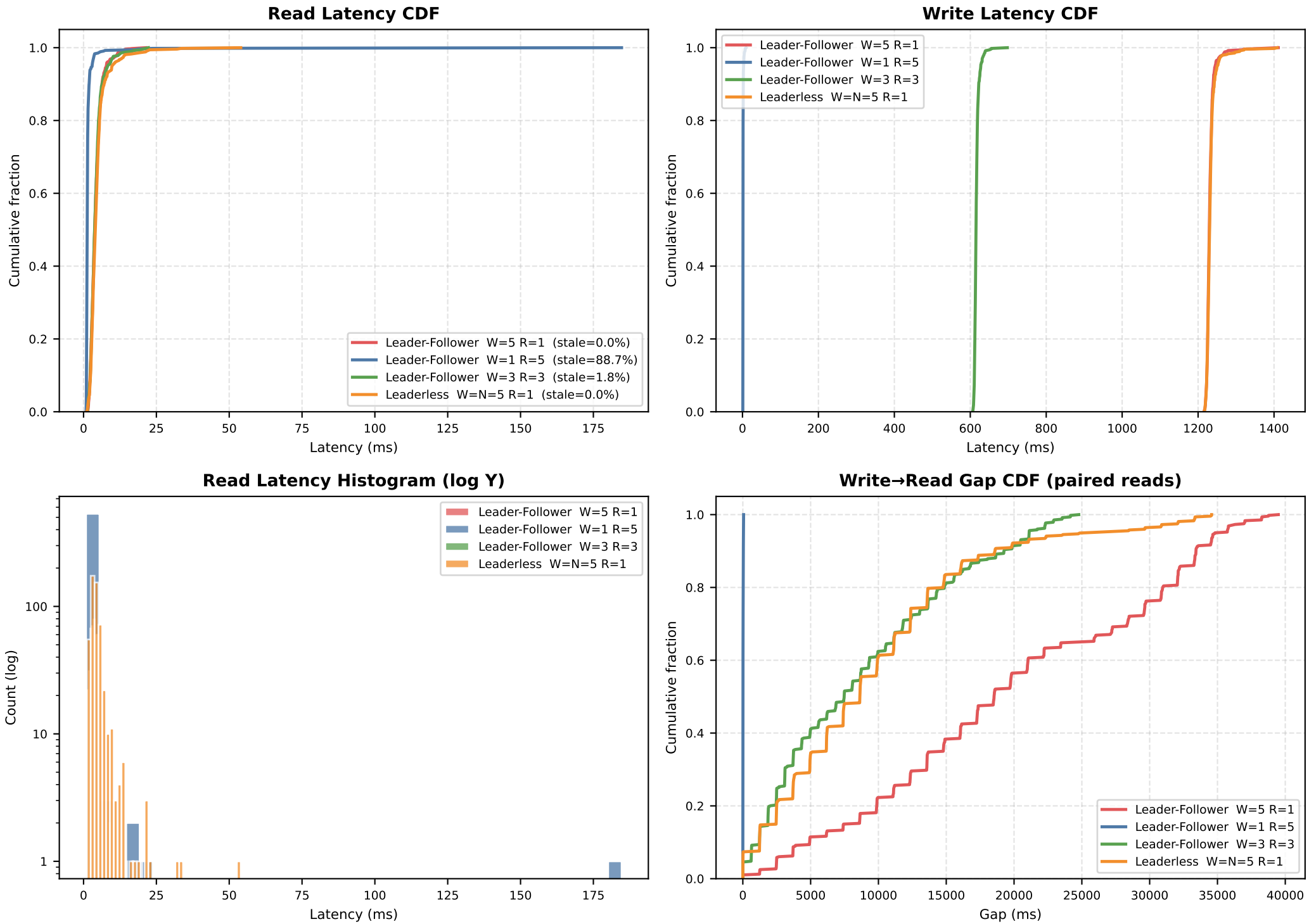
Latency Distributions — 1% writes / 99% reads



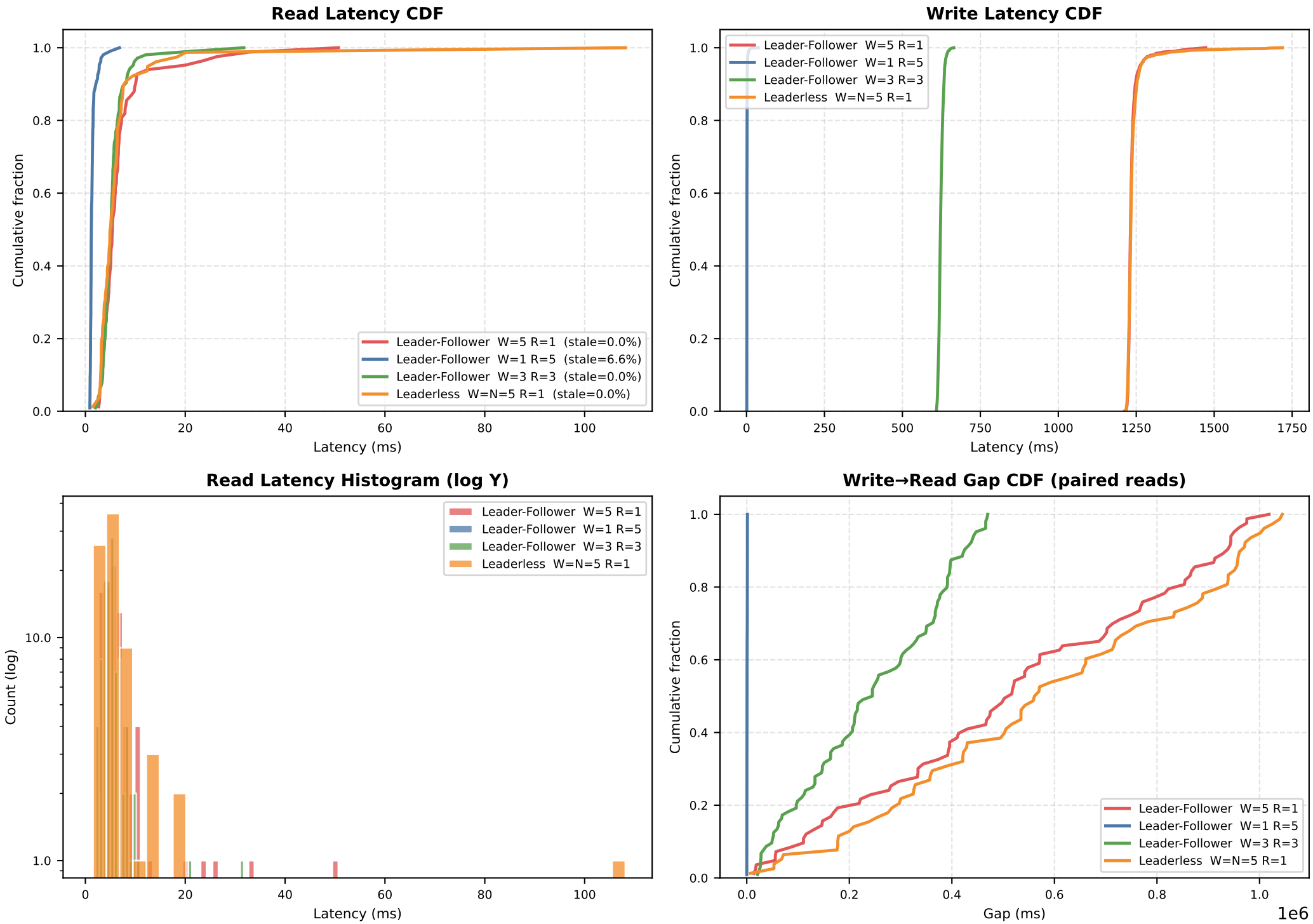
Latency Distributions — 10% writes / 90% reads



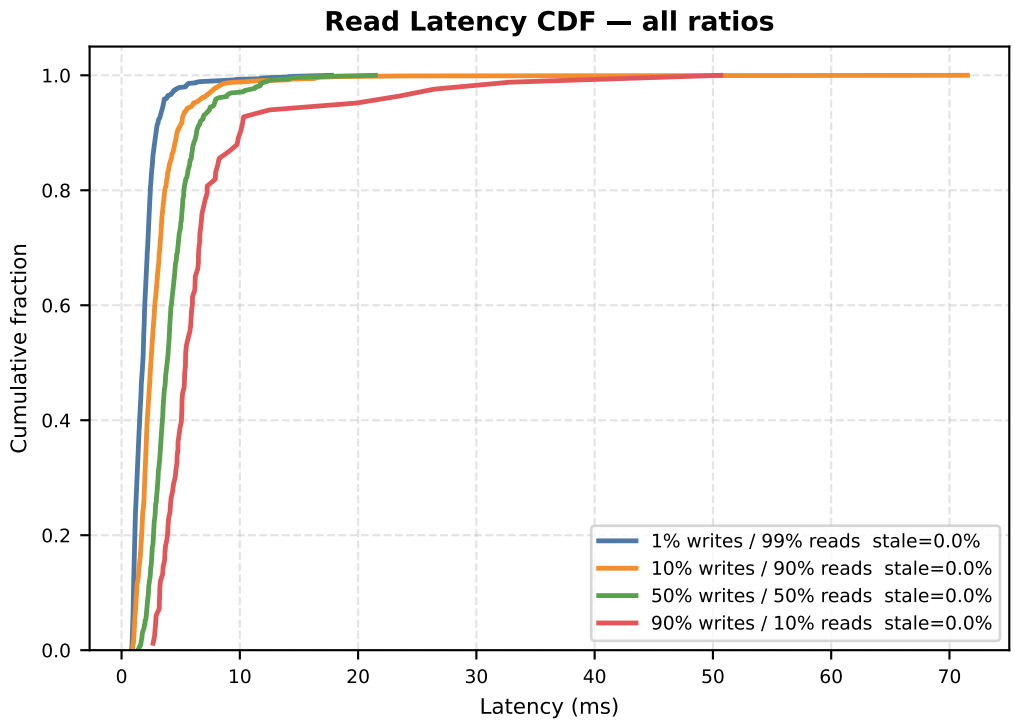
Latency Distributions — 50% writes / 50% reads

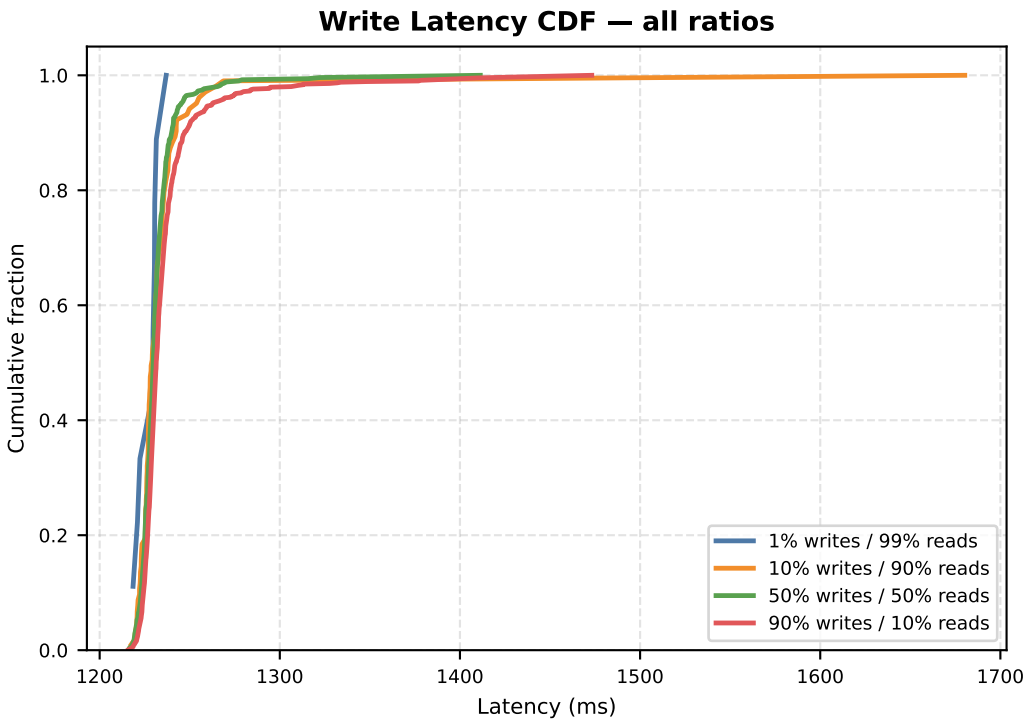


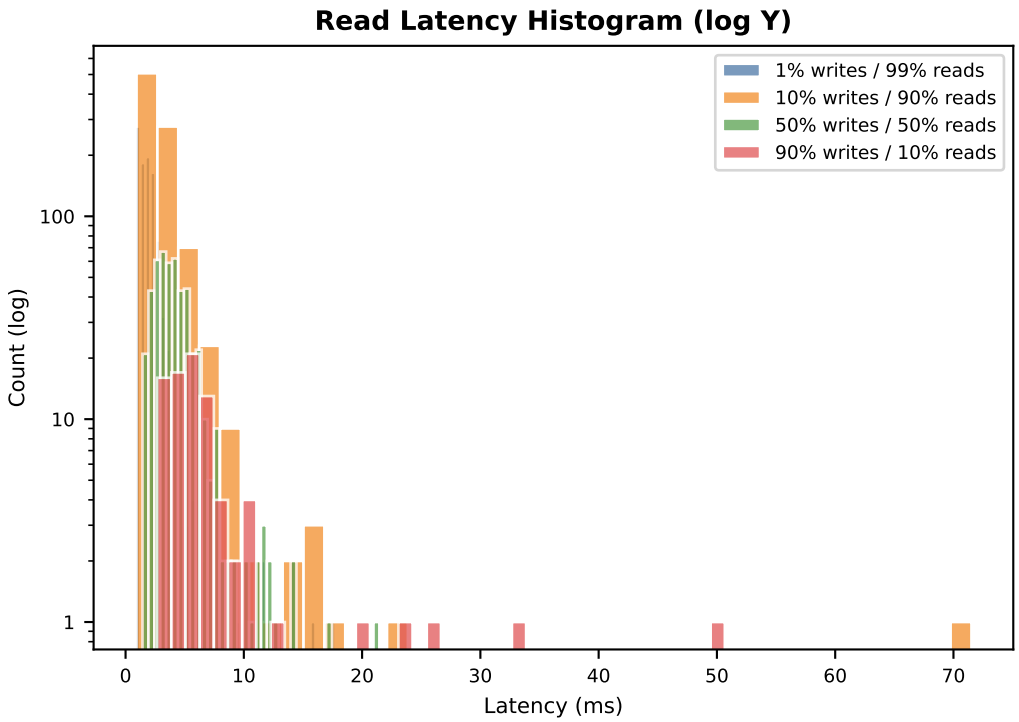
Latency Distributions — 90% writes / 10% reads

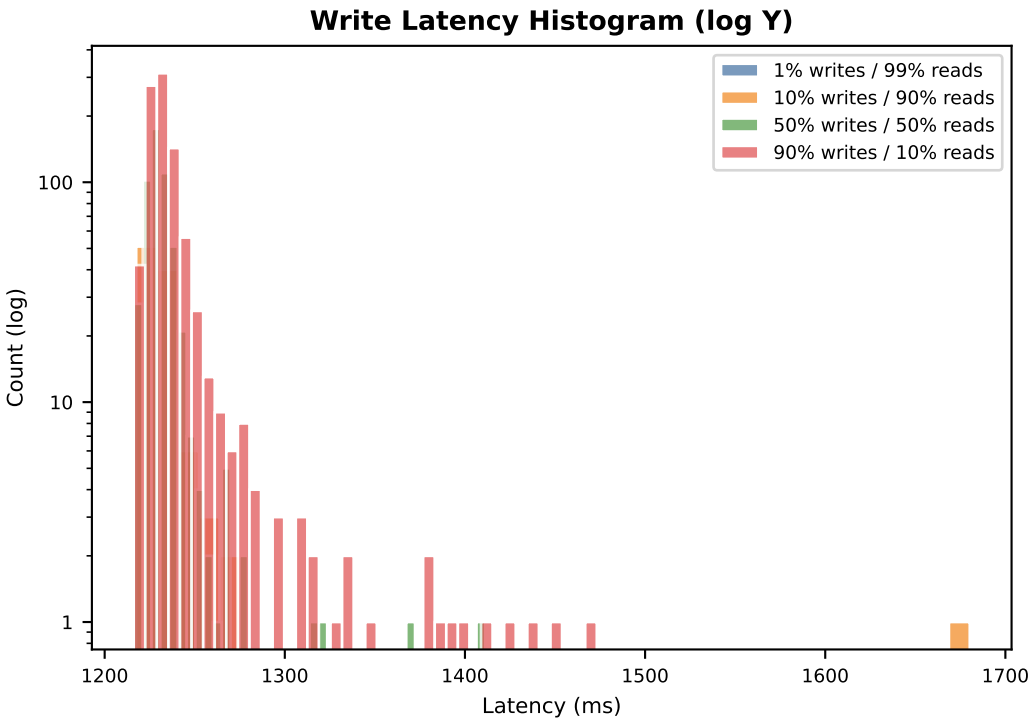


All Ratios — Leader-Follower W=5 R=1

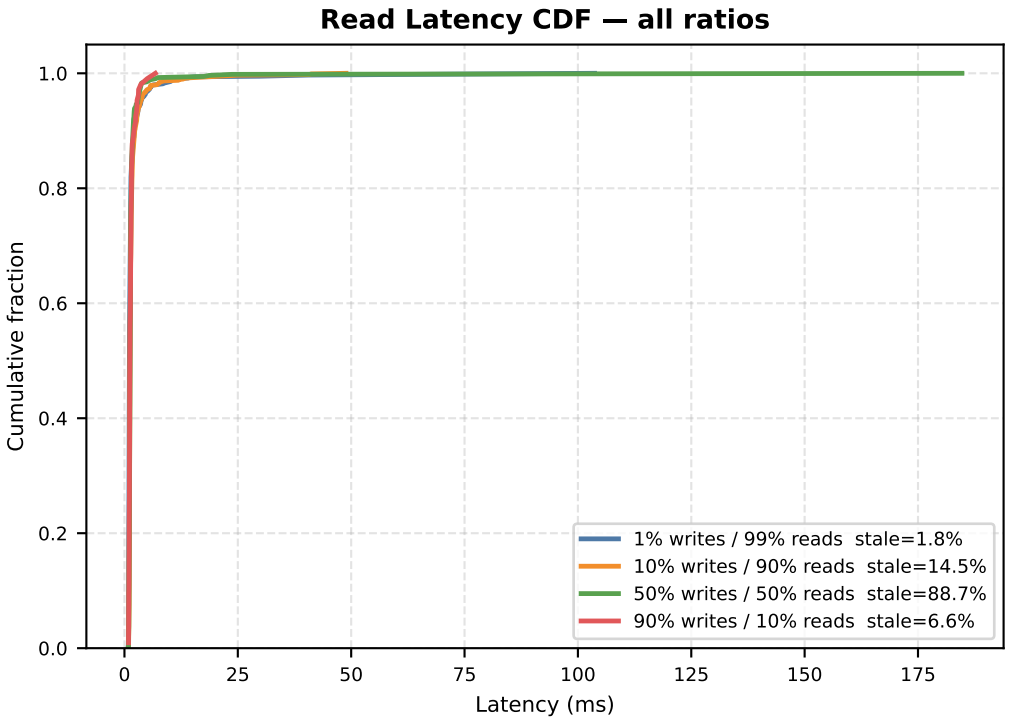


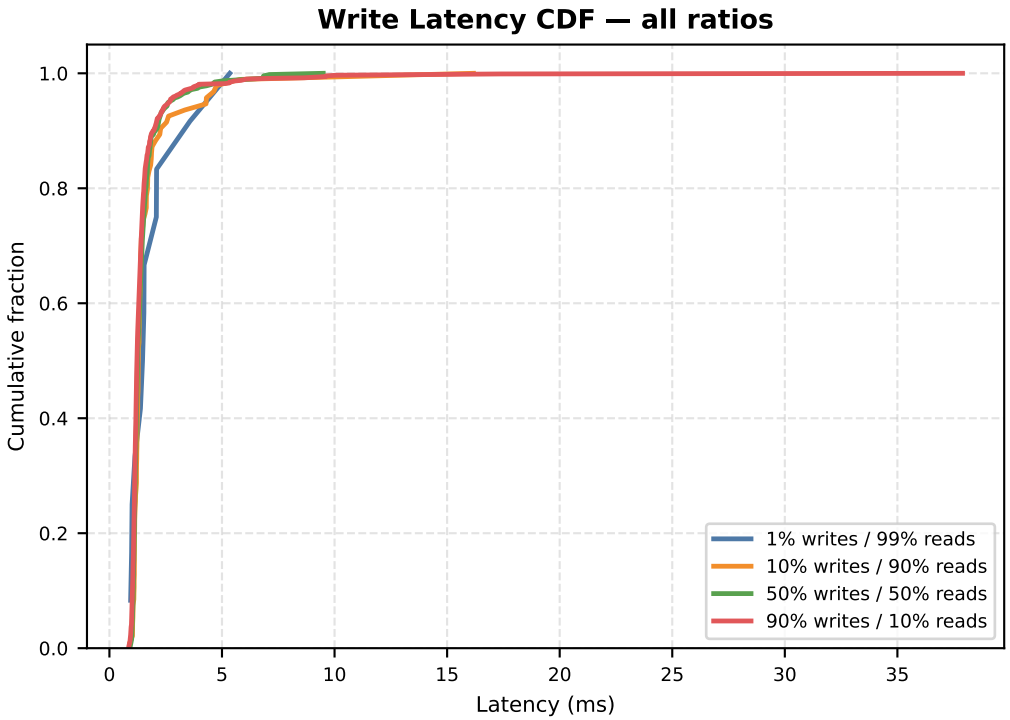


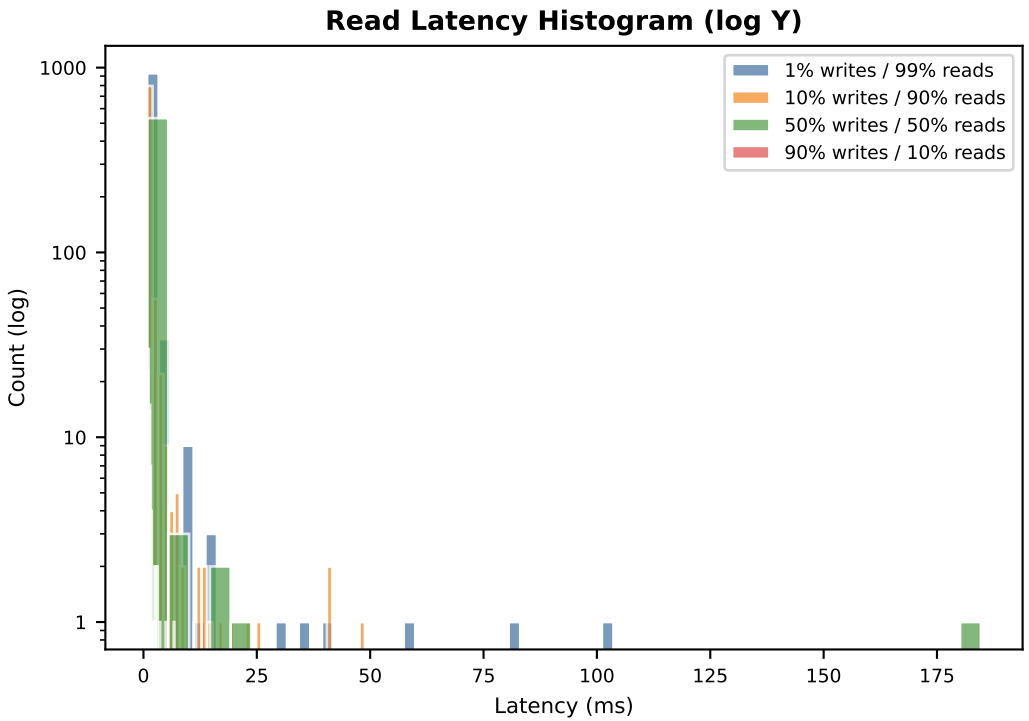


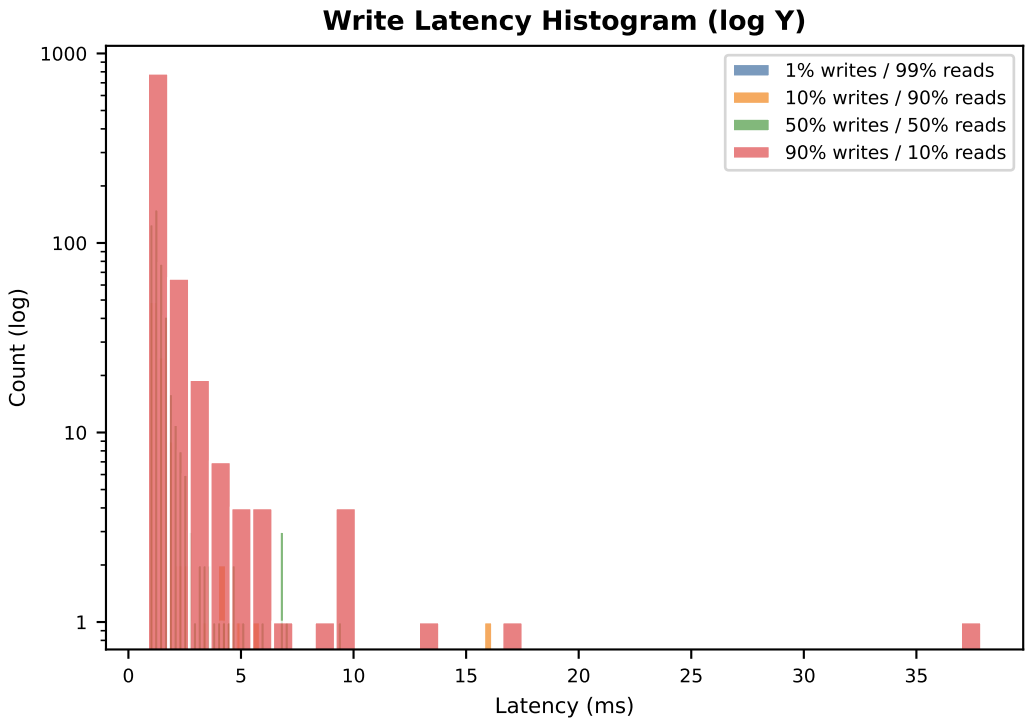


All Ratios — Leader-Follower W=1 R=5

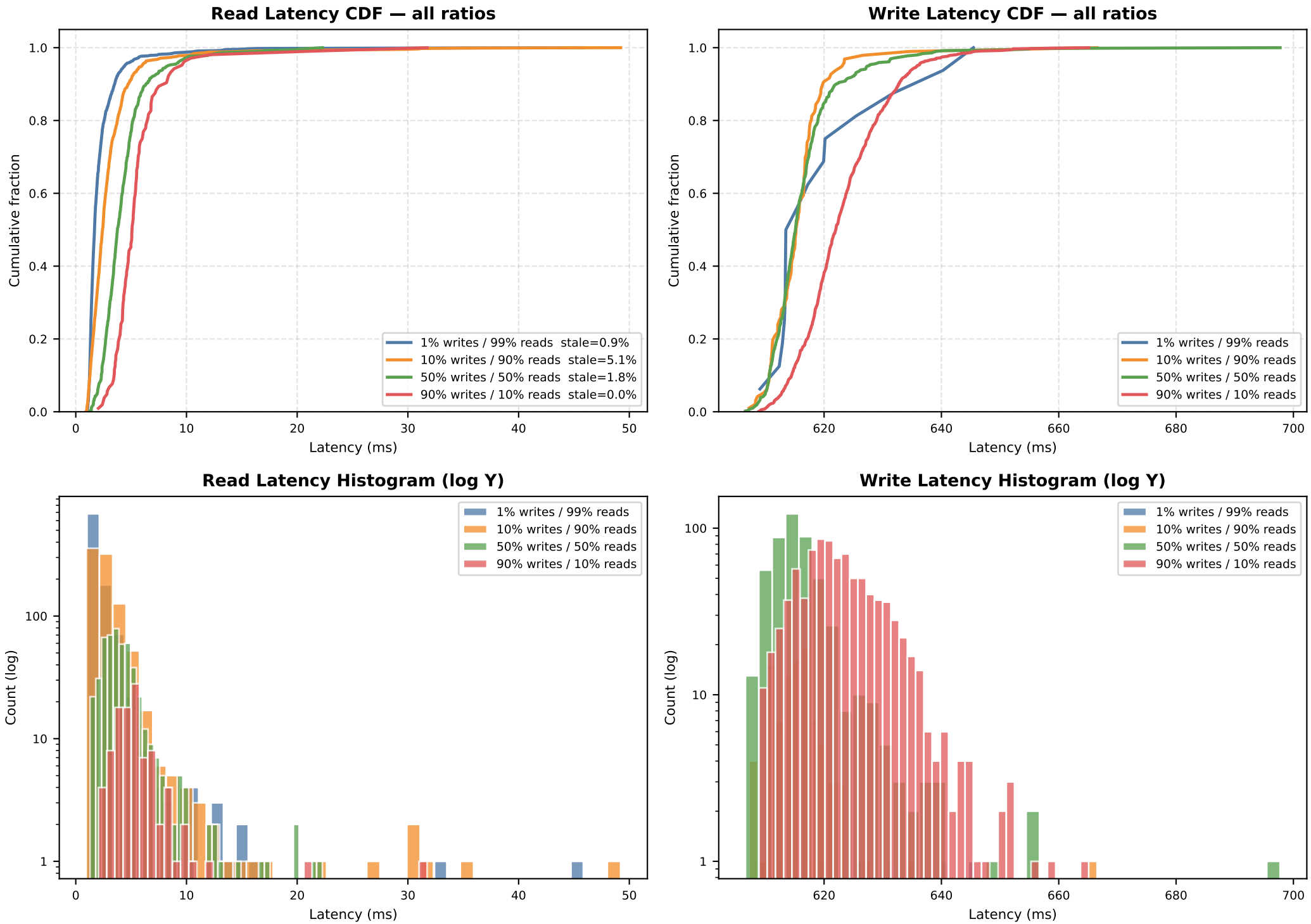




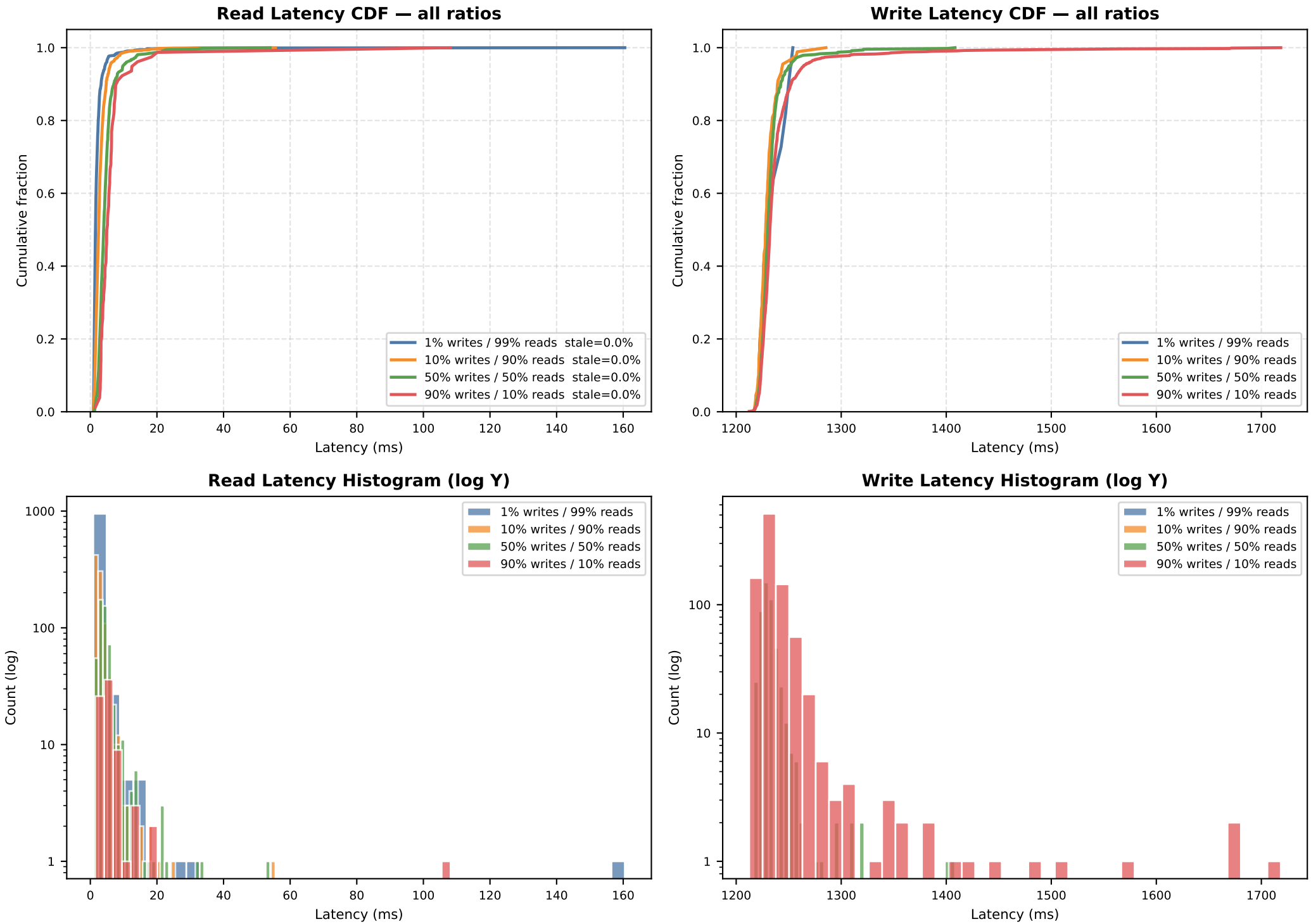




All Ratios — Leader-Follower W=3 R=3



All Ratios — Leaderless W=N=5 R=1



Section 3 — Latency & Staleness Analysis

LATENCY ANALYSIS

W=5 R=1 (write avg ~1,232 ms · read avg ~2–7 ms · stale 0%)
Every write blocks until all 4 followers ACK. Minimum time = 4 followers × (200 ms leader sleep + 100 ms follower sleep) = 1,200 ms. The write histogram shows a sharp spike at ~1,230 ms with almost no tail — the mandatory sleeps dominate variance, not network jitter. Reads return the leader's local value instantly (R=1 = local lookup only). Zero staleness at every ratio because followers are always fully updated before ACK.

W=1 R=5 (write avg ~2 ms · read avg ~1–3 ms · stale 1.8%–88.7%)
Write ACKs after local store only; follower updates are async. Writes take ~2 ms — essentially HTTP overhead. Stale rate grows dramatically with write frequency: 1.8% at 1% writes, 88.7% at 50% writes. At 50/50, writes arrive every ~2 ms but async propagation takes ~1,200 ms, leaving hundreds of un-propagated keys at any moment. The 6.6% at 90% writes is lower because the rare reads mostly occur long after propagation has finished.

W=3 R=3 (write avg ~620 ms · read avg ~2–6 ms · stale 0%–5.1%)
Waits for 2 follower ACKs (self + 2 = quorum of 3). Minimum time = 2 × 300 ms = 600 ms — exactly half the W=5 cost. Because W+R = 6 > N = 5, the write and read quorums always overlap: the leader's R=3 quorum read always includes at least one node from the write quorum, guaranteeing the latest version is returned. Residual 5.1% staleness occurs when clients read directly from follower ports (bypassing the quorum read), hitting one of the 2 followers not in the write quorum.

Leaderless W=N=5 (write avg ~1,238 ms · read avg ~2–8 ms · stale 0%)
Coordinator synchronously updates all 4 peers before returning — identical cost to W=5 leader-follower. Any node can be write coordinator (no single bottleneck). Zero staleness at all ratios. R=1 means each node reads its own local store; since W=N, all nodes have the value after the write ACK.

Section 4 — Best Config per Ratio & Application Guide

WHICH CONFIGURATION WINS AT EACH READ/WRITE RATIO

W=1% / R=99% → Best: W=5 R=1 or Leaderless
Writes are rare so their cost barely matters. W=5 R=1 delivers ~2 ms reads with 0% staleness. W=3 R=3 is also fine (0.9% stale). W=1 R=5 shows only 1.8% stale but its async nature is risky for strong-consistency needs.

W=10% / R=90% → Best: W=5 R=1 or W=3 R=3
W=5 R=1: 0% stale but write path costs 1,232 ms each. W=3 R=3: 5.1% stale but write cost is halved (620 ms) — a good balance. W=1 R=5 at 14.5% stale is risky for applications where reads must reflect recent writes.

W=50% / R=50% → Best: W=3 R=3
W=5 and Leaderless: half the requests are 1.2 s writes — throughput tanks. W=1 R=5: 88.7% stale — nearly every paired read is stale — unacceptable for most applications. W=3 R=3: 1.8% stale at only 620 ms per write. Clear winner for balanced workloads.

W=90% / R=10% → Best: W=1 R=5
Write latency dominates everything. W=1 R=5 writes in ~2 ms with only 6.6% stale reads (rare reads arrive after propagation). W=3 R=3 is second at 620 ms. W=5 and Leaderless are impractical — 900 writes × 1,232 ms each would take ~18 minutes for 1,000 requests.

WHICH DATABASE FOR WHICH APPLICATION

W=5 R=1 → Read-heavy, zero-tolerance for staleness
Banking, medical records, inventory. Writes are rare; reads must always see the latest value. The 1.2 s write is acceptable because writes are infrequent and correctness is paramount.

W=1 R=5 → Write-heavy, eventual consistency acceptable
Social media feeds, IoT sensor ingestion, logging, analytics. High write throughput, occasional stale reads tolerated. Bad choice if users expect to immediately read back what they just wrote.

W=3 R=3 → Balanced load, reasonable consistency required
Shopping carts, session state, collaborative tools. The quorum overlap (W+R > N) provides strong consistency through the leader while keeping write cost half that of W=5. General-purpose choice.

Leaderless W=N → High availability, no single leader bottleneck
Distributed configuration, coordination services. Same consistency as W=5 R=1 but any node can accept writes, improving fault tolerance. Requires all